Robot Code How-To

Making a subsystem, a teleop mode, and autonomous routines you can pick on the driver station

Team 467 · WPILib 2027 (alpha 7) · Commands v3 · AdvantageKit · Systemcore

How the code fits together

Every 20 ms the robot runs the same loop, in this order (see `Robot.robotPeriodic()`):

Step	What happens	Where
1. Read inputs	Each subsystem reads its sensors/motors into an **inputs** object, which AdvantageKit logs.	`updateInputs()`, registered with `Scheduler.addPeriodic`
2. Run commands	The scheduler checks button triggers and runs commands. Commands only change a subsystem's **outputs** object; they never talk to hardware.	`scheduler.run()`
3. Apply outputs	Each subsystem sends its outputs to the motors, once.	`subsystem.applyOutputs()`
4. Log	AdvantageKit writes everything to the log / NetworkTables.	`Logger`

Folder (src/main/java/first/robot/...)	What lives there
`mechanisms/<name>/`	One folder per subsystem: IO interface, hardware IO, sim IO, constants, the Mechanism class
`opmodes/teleop/`	Teleop modes and their button bindings (`TeleopBindings`)
`opmodes/auto/`, `commands/auto/`	Autonomous opmodes and the `Autos` routines
`Orchestrator.java`	Commands that use several subsystems together (shooting, aiming, feeding)
`Robot.java`	Creates every subsystem, sets default commands, registers autos, runs the loop
`Schematic.java`	CAN IDs and the CAN bus for every device
`src/main/deploy/pathplanner/`	PathPlanner paths, autos and settings (copied to the robot on deploy)

Part 1 — Making a subsystem

A subsystem is split into an **IO layer** (the only code that touches hardware) and a **Mechanism** (state and commands). The same Mechanism runs with real hardware, in simulation, or replaying a log, just by handing it a different IO. This walkthrough builds a **Climber** with one SPARK MAX. Every file below compiles against this project.

File	Job
`ClimberConstants`	Numbers: voltages, limits, inversion
`ClimberIO`	Defines the inputs (what we read) and outputs (what we want). No hardware.
`ClimberIOSparkMax`	Real hardware: configures the motor, reads it, drives it
`ClimberIOSim`	Fake hardware for simulation
`Climber`	The Mechanism: commands that set outputs

Create the folder `mechanisms/climber/` and add these files.

Step 1 — Constants

`mechanisms/climber/ClimberConstants.java`

```
package first.robot.mechanisms.climber;

public final class ClimberConstants {
  public static final boolean MOTOR_INVERTED = false;
  public static final int CURRENT_LIMIT = 40; // amps
  public static final double CLIMB_VOLTS = 8.0;
  public static final double LOWER_VOLTS = -4.0;

  private ClimberConstants() {}
}
```

Step 2 — The IO interface

Inputs are plain fields with `@AutoLog` (AdvantageKit generates `ClimberIOInputsAutoLogged` at build time). Outputs hold a **mode** plus targets, and must default to `STOP` so nothing moves before a command asks it to.

`mechanisms/climber/ClimberIO.java`

```
package first.robot.mechanisms.climber;

import org.littletonrobotics.junction.AutoLog;
import org.littletonrobotics.junction.AutoLogOutput;

public interface ClimberIO {
  /** Everything read from hardware. AdvantageKit logs it and can replay it. */
  @AutoLog
  public static class ClimberIOInputs {
    public boolean connected = false;
    public double positionRotations = 0.0; // motor rotations
    public double appliedVolts = 0.0;
    public double currentAmps = 0.0;
  }

  public default void updateInputs(ClimberIOInputs inputs) {}

  /** The ways the mechanism can be driven. */
  public static enum ClimberIOOutputMode {
    VOLTAGE,
    STOP
  }

  /** What commands want this loop. Always starts as STOP. */
  public static class ClimberIOOutputs {
    @AutoLogOutput(key = "Climber/Output Mode")
    public ClimberIOOutputMode mode = ClimberIOOutputMode.STOP;

    @AutoLogOutput(key = "Climber/Target Volts")
    public double targetVolts = 0.0;
  }

  public default void applyOutputs(ClimberIOOutputs outputs) {}
}
```

Step 3 — Hardware IO

Put all motor configuration here. Two 2027 rules: take the CAN port and ID from `Schematic`, and remember REVLib 2027 getters return a **Signal** (use `.get()` and `.isValid()`) and there are no conversion factors (convert units yourself).

`mechanisms/climber/ClimberIOSparkMax.java`

```java
package first.robot.mechanisms.climber;

import static first.robot.mechanisms.climber.ClimberConstants.*;

import com.revrobotics.PersistMode;
import com.revrobotics.RelativeEncoder;
import com.revrobotics.ResetMode;
import com.revrobotics.spark.SparkLowLevel.MotorType;
import com.revrobotics.spark.SparkMax;
import com.revrobotics.spark.config.SparkBaseConfig.IdleMode;
import com.revrobotics.spark.config.SparkMaxConfig;
import first.robot.Schematic;

/** Real hardware: one SPARK MAX + NEO. */
public class ClimberIOSparkMax implements ClimberIO {
  private final SparkMax motor;
  private final RelativeEncoder encoder;

  public ClimberIOSparkMax() {
    // CAN port and ID always come from Schematic
    motor = new SparkMax(Schematic.canPort, Schematic.climberMotorCanId, MotorType.kBrushless);

    var config = new SparkMaxConfig();
    config.inverted(MOTOR_INVERTED).idleMode(IdleMode.kBrake).smartCurrentLimit(CURRENT_LIMIT);
    motor.configure(config, ResetMode.kResetSafeParameters, PersistMode.kPersistParameters);

    encoder = motor.getEncoder();
  }

  @Override
  public void updateInputs(ClimberIOInputs inputs) {
    // REVLib 2027 getters return a Signal: .get() for the value, .isValid() if it's fresh
    var position = encoder.getPosition();
    var output = motor.getAppliedOutput();
    var bus = motor.getBusVoltage();
    var current = motor.getOutputCurrent();
    inputs.connected = position.isValid() && output.isValid() && bus.isValid();
    inputs.positionRotations = position.get();
    inputs.appliedVolts = output.get() * bus.get();
    inputs.currentAmps = current.get();
  }

  @Override
  public void applyOutputs(ClimberIOOutputs outputs) {
    switch (outputs.mode) {
      case VOLTAGE -> motor.setVoltage(outputs.targetVolts);
      case STOP -> motor.stopMotor();
    }
  }
}
```

Note: Add the CAN ID to `Schematic.java` for every robot type before using it. `climberMotorCanId` already exists (22 on the 2026 comp robot). For a TalonFX, use `new TalonFX(id, Schematic.canBus)` instead (see `IntakeRollerIOTalonFX`).

Step 4 — Sim IO

Just enough physics that commands behave sensibly in simulation. It doesn't need to be accurate.

`mechanisms/climber/ClimberIOSim.java`

```
package first.robot.mechanisms.climber;

/** Simple sim: position moves at a speed proportional to volts. */
public class ClimberIOSim implements ClimberIO {
  private static final double DT = 0.02;
  private static final double ROTATIONS_PER_SEC_PER_VOLT = 2.0; // placeholder

  private double position = 0.0;
  private double appliedVolts = 0.0;

  @Override
  public void updateInputs(ClimberIOInputs inputs) {
    inputs.connected = true;
    inputs.positionRotations = position;
    inputs.appliedVolts = appliedVolts;
    inputs.currentAmps = Math.abs(appliedVolts) * 2.0;
  }

  @Override
  public void applyOutputs(ClimberIOOutputs outputs) {
    appliedVolts = outputs.mode == ClimberIOOutputMode.VOLTAGE ? outputs.targetVolts : 0.0;
    position += appliedVolts * ROTATIONS_PER_SEC_PER_VOLT * DT;
  }
}
```

Step 5 — The Mechanism

Commands are coroutines. A command that should keep running (hold a voltage, hold a position) sets outputs and calls `coroutine.park()`. A command that finishes waits for its goal with `coroutine.waitUntil(...)` and then returns. Override `idle()` so the mechanism is safe whenever no command is using it.

`mechanisms/climber/Climber.java`

```
package first.robot.mechanisms.climber;

import static first.robot.mechanisms.climber.ClimberConstants.*;

import first.robot.mechanisms.climber.ClimberIO.ClimberIOOutputMode;
import first.robot.mechanisms.climber.ClimberIO.ClimberIOOutputs;
import org.littletonrobotics.junction.Logger;
import org.wpilib.command3.Command;
import org.wpilib.command3.Mechanism;
import org.wpilib.command3.Scheduler;

public class Climber implements Mechanism {
  private final ClimberIO io;
  private final ClimberIOInputsAutoLogged inputs = new ClimberIOInputsAutoLogged();
  private final ClimberIOOutputs outputs = new ClimberIOOutputs();

  public Climber(ClimberIO io) {
    this.io = io;
    // Read inputs at the start of every Scheduler.run(), before commands run
    Scheduler.getDefault().addPeriodic(this::updateInputs);
  }

  private void updateInputs() {
    io.updateInputs(inputs);
    Logger.processInputs("Climber", inputs);
  }

  /** Call from Robot.robotPeriodic() right after the scheduler runs. */
  public void applyOutputs() {
    io.applyOutputs(outputs);
  }

  private void runVoltage(double volts) {
    outputs.mode = ClimberIOOutputMode.VOLTAGE;
    outputs.targetVolts = volts;
  }

  private void stopOutput() {
    outputs.mode = ClimberIOOutputMode.STOP;
  }

  public double getPosition() {
    return inputs.positionRotations;
  }

  /** Default command: stopped whenever nothing else is using the climber. */
  @Override
  public Command idle() {
    return run(coroutine -> {
          stopOutput();
```

```
          coroutine.park();
        })
        .withPriority(Command.LOWEST_PRIORITY)
        .named("Climber.Idle");
  }

  /** Climbs while running. Stops when interrupted (the idle command takes over). */
  public Command climb() {
    return run(coroutine -> {
          runVoltage(CLIMB_VOLTS);
          coroutine.park();
        })
        .named("Climber.Climb");
  }

  /** Lowers until the climber reaches the given position, then ends. */
  public Command lowerTo(double positionRotations) {
    return run(coroutine -> {
          runVoltage(LOWER_VOLTS);
          coroutine.waitUntil(() -> getPosition() <= positionRotations);
          stopOutput();
        })
        .named("Climber.LowerTo");
  }
}
```

Step 6 — Wire it into Robot.java (four places)

`Robot.java`

```
// (a) a field
@NotLogged public final Climber climber;

// (b) in the constructor's switch, one line per mode
case REAL -> { ...  climber = new Climber(new ClimberIOSparkMax()); }
case SIM ->  { ...  climber = new Climber(new ClimberIOSim()); }
default ->   { ...  climber = new Climber(new ClimberIO() {}); }   // replay: no hardware

// (c) its default command, with the others
climber.setDefaultCommand(climber.idle());

// (d) in robotPeriodic(), after scheduler.run(), with the other applyOutputs() calls
climber.applyOutputs();
```

Watch out: Forgetting (d) is the most common mistake: commands will run and the logs will show the right “Target Volts”, but the motor never moves.

Step 7 — Test it

- Run **WPILib: Simulate Robot Code** (or `gradlew simulateJava`) and connect AdvantageScope to *localhost*.
- Check `Climber/...` (inputs) and `Climber/Output Mode`, `Climber/Target Volts` (outputs) while you press the button you bound.
- On the robot, put it on blocks first and confirm direction, current limit and `connected` before driving it hard.

Part 2 — Making a teleop mode

In WPILib 2027 the driver station shows a list of **opmodes**; you pick one for teleop and one for autonomous before enabling. An opmode is a class whose constructor sets up button bindings and default commands. Everything it sets up only lasts while that opmode is selected, then reverts automatically.

There are two teleop opmodes now: **DefaultTeleop** (normal) and **Manual** (manual intake arm). Both share most bindings through `TeleopBindings`.

Option A — Add a button to the existing modes

Add one line in `opmodes/teleop/TeleopBindings.java`: in `bindCommon` for both modes, `bindAutoMode` for DefaultTeleop only, or `bindManualMode` for Manual only.

```
// in TeleopBindings.bindCommon(Robot robot):
operator.faceDown().whileTrue(robot.climber.climb());   // climb while operator A is held
```

Option B — A new teleop mode

Create a class in `opmodes/teleop/` that implements `OpMode`, is annotated with `@Teleop`, and has a constructor taking `Robot`. The robot finds it automatically on startup; there is nothing to register.

`opmodes/teleop/PracticeTeleop.java`

```
package first.robot.opmodes.teleop;

import first.robot.Robot;
import org.wpilib.command3.button.CommandGamepad;
import org.wpilib.opmode.OpMode;
import org.wpilib.opmode.Teleop;

/** Shows up on the driver station as "Practice" under the "Testing" group. */
@Teleop(name = "Practice", group = "Testing", description = "Drive only, slow intake")
public class PracticeTeleop implements OpMode {
  public PracticeTeleop(Robot robot) {
    CommandGamepad driver = robot.driverController;

    // Default commands set here only apply while this opmode is selected
    robot.drive.setDefaultCommand(robot.drive.driverControl(driver));

    driver.start().onTrue(robot.drive.resetHeading()); // run once per press
    driver.leftTrigger(0.2).whileTrue(robot.intake.runIntakeMotor()); // run while held
    driver.faceDown().toggleOnTrue(robot.drive.lock()); // press on, press off
  }
}
```

Button names

The 2027 `CommandGamepad` uses position names instead of Xbox letters. Driver is port 0, operator port 1 (`robot.driverController`, `robot.operatorController`).

Xbox	CommandGamepad	Xbox	CommandGamepad
A	`faceDown()`	LB / RB	`leftBumper()` / `rightBumper()`
B	`faceRight()`	LT / RT	`leftTrigger(0.2)` / `rightTrigger(0.2)`
X	`faceLeft()`	D-pad	`dpadUp()`, `dpadDown()`, `dpadLeft()`, `dpadRight()`
Y	`faceUp()`	Stick clicks	`leftStick()` / `rightStick()`
Start / Back	`start()` / `back()`	Sticks	`getLeftX()`, `getLeftY()`, `getRightX()`

Binding types

Method	Behavior	Typical use
`onTrue(cmd)`	Starts the command when pressed; it runs until it ends or is interrupted	Reset heading, home a mechanism
`whileTrue(cmd)`	Runs while held, cancels on release	Intake rollers, hold-to-shoot
`toggleOnTrue(cmd)`	Press to start, press again to cancel	Spin up shooter, drive to hub
`onFalse(cmd)`	Starts the command when released	Stop something after a hold
`a.and(b)`, `a.negate()`	Combine triggers (button combos, modifiers)	`rightTrigger(0.1).and(dpadUp())`
`new Trigger(() -> ...)`	A trigger from any condition	new `Trigger(intake::isIntaking)` for rumble

Note: Two commands that use the same subsystem can't run at once: starting one cancels the other, and when nothing uses a subsystem its default command (usually `idle()`) runs. Multi-subsystem actions belong in `Orchestrator` so the buttons stay one-liners.

Part 3 — Adding autonomous routines

Each auto is an autonomous **opmode**, so it appears in the driver station's autonomous list, grouped by name. There are three ways to add one; pick the simplest that fits.

Way	Best for	Appears on the DS as
A. Annotated class	Small, self-contained autos	Its `@Autonomous(name, group)`
B. Routine in `Autos` + one line in `Robot`	Most autos: sequences of paths, shooting and intake	The name/group you pass to `addCommandOpMode`
C. PathPlanner .auto file	Autos built entirely in the PathPlanner app	The file name, in the "PathPlanner" group

Way A — An annotated auto class

Same idea as a teleop mode, with `@Autonomous`. Build the command in the constructor and start it when the robot is enabled. Found automatically; nothing to register.

`opmodes/auto/DriveForwardAuto.java`

```
package first.robot.opmodes.auto;

import static org.wpilib.units.Units.Seconds;

import first.robot.Robot;
import org.wpilib.command3.Command;
import org.wpilib.command3.Trigger;
import org.wpilib.driverstation.RobotState;
import org.wpilib.math.kinematics.ChassisVelocities;
import org.wpilib.opmode.Autonomous;
import org.wpilib.opmode.OpMode;

/** Drives forward at 1 m/s for 2 seconds. Found automatically because of @Autonomous. */
@Autonomous(name = "Drive Forward", group = "Simple")
public class DriveForwardAuto implements OpMode {
  private final Trigger enabled = new Trigger(RobotState::isEnabled);

  public DriveForwardAuto(Robot robot) {
    Command driveForward =
        robot
            .drive
            .driveRobotRelative(() -> new ChassisVelocities(1.0, 0.0, 0.0))
            .withTimeout(Seconds.of(2.0));
    enabled.onTrue(driveForward);
  }
}
```

Way B — A routine in Autos (recommended)

`commands/auto/Autos.java` has helpers that read like the 2026 `Commands.*` calls, plus access to drive, the Orchestrator, intake and shooter. Write a method that returns a `Command`:

`commands/auto/Autos.java`

```
  /** Shoots the preload from the hub, then follows the A-Cycle1 path out. */
  public Command shootThenLeave() {
    return sequence(
        timeout(EightBalls(), 6.0), // drive to hub, spin up, feed
        timeout(orchestrator.stopShootingAuto(), 0.1),
        deadline(
            drive.followPath("A-Cycle1"), // file: deploy/pathplanner/paths/A-Cycle1.path
            intake.extendToAngleAndIntake(IntakeConstants.EXTEND_POS)));
  }
```

Then register it in `Robot.registerCommandOpModes()` (name, group, method):

`Robot.java`

```
addCommandOpMode("Shoot Then Leave", "Manual Autos", autos::shootThenLeave);
```

Helper (in Autos)	Does
`sequence(a, b, c)`	Run one after another
`parallel(a, b)`	Run together; ends when all end
`deadline(main, others...)`	Run together; ends when *main* ends (others are cancelled)
`timeout(cmd, seconds)`	Cancel `cmd` if it runs longer than that
`after(seconds, cmd)`, `waitSeconds(s)`	Delay, then run / just wait
`repeatedly(() -> cmd)`	Restart a fresh copy every time it ends
`drive.followPath("Name")`	Follow `deploy/pathplanner/paths/Name.path` (skipped with an error if missing)
`resetPose(supplier)`	Set the starting pose (use `AllianceFlipUtil.apply` for blue-side poses)

Watch out: Commands running together in `parallel`/`deadline` must not use the same subsystem (for example two intake commands). The robot throws an error when that auto is *selected*, so test every new auto by selecting it in sim.

Way C — A PathPlanner auto

- Open the project folder in the PathPlanner app. Paths live in `src/main/deploy/pathplanner/paths/` and autos in `.../autos/`; robot settings are already in `settings.json`.
- Build the auto in the app. Any *.auto* file becomes an autonomous opmode on the next deploy, in the “PathPlanner” group; no code needed.
- To run robot actions from the app (event markers / named commands), use the names registered in `Robot.registerPathPlannerAutos()`: `Intake`, `Stow`, `HomeIntake`, `RunMagicCarpet`, `RunIndexer`, `ReverseIndexer`, `ShootAtHub`. Add more with:

```
NamedCommands.registerCommand("Climb", climber.climb());   // before the autos are registered
```

Picking an auto at the match

- Deploy (`gradlew deploy` or **WPILib: Deploy Robot Code**). New paths and .auto files go with it.
- On the driver station, choose the autonomous opmode (grouped: *Manual Autos*, *PathPlanner Cycles*, *PathPlanner*, *Simple*, *Characterization*, *Utility*) and the teleop opmode.
- Autos are built when you **select** them, so a broken auto shows its error then, before the match starts.

Checklist and common mistakes

Symptom	Likely cause / fix
Motor never moves, but logs show the target	`applyOutputs()` isn't called in `Robot.robotPeriodic()`.
Motor not found / `connected` false	Wrong CAN ID in `Schematic`, or not using `Schematic.canPort` / `canBus` (CAN_S ports are Systemcore; CAN_D are Motioncore).
Robot freezes when a command runs	A loop inside a command without `coroutine.yield()`. Every loop must yield.
Command ends instantly and the mechanism stops	It didn't `park()`; the default command took over the next loop.
Error when selecting an auto	Two commands in the same parallel/deadline use the same subsystem, or a constructor throws.
Path step skipped	Path file name doesn't match `drive.followPath("...")` exactly (check `deploy/pathplanner/paths`).
Wrong units	REVLib 2027 has no conversion factors: encoder position is motor rotations, velocity is RPM.
`...AutoLogged` not found	Build once; AdvantageKit generates it from the `@AutoLog` class.

New subsystem checklist

- CAN ID added to `Schematic` for every robot type
- `XIO` (inputs with `connected`, outputs defaulting to STOP), `XIOSparkMax`/`XIOTalonFX`, `XIOSim`
- `X` Mechanism: `addPeriodic(updateInputs)` in the constructor, `applyOutputs()`, `idle()` override
- Robot: field, three IO cases, default command, `applyOutputs()` call
- Buttons in `TeleopBindings` or a new opmode; autos via `Autos` + `addCommandOpMode`
- Tested in sim with AdvantageScope, then on blocks on the robot